

Introducción a la Programación con Python

Semana 8: Programación Orientada a Objetos

∇Nabla labs

Contenido

- 1 ¿Qué es la POO?
- 2 Clases y objetos
- 3 Método `__init__` (constructor)
- 4 Atributos de instancia y de clase
- 5 Métodos de instancia
- 6 Encapsulación y convenciones
- 7 Propiedades con `@property`
- 8 Métodos especiales
- 9 Herencia
- 10 Métodos de clase
- 11 Métodos estáticos
- 12 Proyecto: Sistema de gestión

¿Qué es POO?

-  Paradigma de programación basado en **objetos**
 -  Objetos combinan **datos** (atributos) y **comportamiento** (métodos)
 -  Organiza código de forma modular y reutilizable
 -  Modela conceptos del mundo real
- Encapsula datos y funcionalidad

 ¡Piensa en objetos, no solo en funciones!

¿Por qué usar POO? ?

Sin POO: ❌

- Datos dispersos
- Funciones aisladas
- Difícil de mantener
- Código repetitivo
- Menos organización

Con POO: ✔

- Datos agrupados
- Comportamiento asociado
- Fácil de extender
- Código reutilizable
- Mejor organización

i POO facilita modelar sistemas complejos

¿Qué es una clase?

Una **clase** es un molde o plantilla para crear objetos. Define qué atributos y métodos tendrán los objetos.

Ejemplo conceptual:

 **Clase:** Perro (el concepto general)

 **Objeto:** Mi perro "Max" (una instancia específica)

Sintaxis básica:

```
1 class NombreClase:
2     # Definición de la clase
3     pass
```

 Por convención, los nombres de clases usan CamelCase

Primera clase simple </>

```
1 class Perro:
2     # Atributos de clase
3     especie = "Canis familiaris"
4
5     # Método simple
6     def ladrar(self):
7         return "¡Guau guau!"
8
9 # Crear objetos (instancias)
10 mi_perro = Perro()
11 tu_perro = Perro()
12
13 print(mi_perro.especie)
14 print(mi_perro.ladrar())
```

➔ Salida

```
Canis familiaris
¡Guau guau!
```

¿Qué es self?

self es una referencia a la instancia actual del objeto. Permite acceder a sus atributos y métodos.

```
1 class Persona:
2     def saludar(self):
3         return "Hola, soy una persona"
4
5     def presentarse(self, nombre):
6         # self permite acceder al mismo objeto
7         return f"{self.saludar()}, me llamo {nombre}"
8
9 persona = Persona()
10 print(persona.presentarse("Ana"))
```

➔ Salida

Hola, soy una persona, me llamo Ana



self debe ser el primer parámetro de los métodos de instancia

¿Qué es `__init__`?

El **constructor** es un método especial que se ejecuta automáticamente cuando se crea un objeto. Inicializa los atributos.

```
1 class Persona:
2     def __init__(self, nombre, edad):
3         self.nombre = nombre    # Atributo de instancia
4         self.edad = edad        # Atributo de instancia
5
6     def presentarse(self):
7         return f"Hola, soy {self.nombre} y tengo {self.edad} años"
8
9 # Crear objetos con valores iniciales
10 ana = Persona("Ana", 25)
11 luis = Persona("Luis", 30)
12
13 print(ana.presentarse())
14 print(luis.presentarse())
```


➔ Salida

```
Hola, soy Ana y tengo 25 años
```

```
Hola, soy Luis y tengo 30 años
```

💡 Cada objeto tiene sus propios valores de atributos

¿Qué son?

Atributos **únicos** para cada objeto. Se definen en `__init__` con `self`.

```
1 class Coche:
2     def __init__(self, marca, modelo, año):
3         # Atributos de instancia
4         self.marca = marca
5         self.modelo = modelo
6         self.año = año
7         self.kilometraje = 0 # Valor inicial por defecto
8
9 coche1 = Coche("Toyota", "Corolla", 2020)
10 coche2 = Coche("Honda", "Civic", 2021)
11
12 print(f"{coche1.marca} {coche1.modelo}")
13 print(f"{coche2.marca} {coche2.modelo}")
```

➡ Salida

Toyota Corolla
Honda Civic

¿Qué son?

Atributos **compartidos** por todas las instancias de la clase. Se definen fuera de `__init__`.

```
1 class Empleado:
2     # Atributo de clase (compartido)
3     empresa = "TechCorp"
4     total_empleados = 0
5
6     def __init__(self, nombre, salario):
7         # Atributos de instancia (únicos)
8         self.nombre = nombre
9         self.salario = salario
10        Empleado.total_empleados += 1
11
12 emp1 = Empleado("Ana", 50000)
13 emp2 = Empleado("Luis", 60000)
14
15 print(f"Empresa: {Empleado.empresa}")
16 print(f"Total empleados: {Empleado.total_empleados}")
```

➡ Salida

```
Empresa: TechCorp  
Total empleados: 2
```

i Los atributos de clase se acceden con `NombreClase.atributo`

¿Qué son?

Funciones definidas dentro de una clase que operan sobre los atributos de la instancia. Siempre llevan **self** como primer parámetro.

```
1 class CuentaBancaria:
2     def __init__(self, titular, saldo=0):
3         self.titular = titular
4         self.saldo = saldo
5
6     def depositar(self, monto):
7         self.saldo += monto
8         return f"Depósito exitoso. Saldo: ${self.saldo}"
9
10    def retirar(self, monto):
11        if monto <= self.saldo:
12            self.saldo -= monto
13            return f"Retiro exitoso. Saldo: ${self.saldo}"
14        return "Fondos insuficientes"
```

Métodos de instancia: Uso

```
1 # Crear cuenta
2 cuenta = CuentaBancaria("Ana García", 1000)
3
4 print(cuenta.depositar(500))
5 print(cuenta.retirar(300))
6 print(cuenta.retirar(2000))
7 print(f"Saldo final: ${cuenta.saldo}")
```

➔ Salida

```
Depósito exitoso. Saldo: $1500
Retiro exitoso. Saldo: $1200
Fondos insuficientes
Saldo final: $1200
```

¿Qué es encapsulación?

Ocultar los detalles internos de un objeto y exponer solo lo necesario. Protege los datos de modificaciones no deseadas.

Convenciones en Python:

 nombre: Público (accesible desde cualquier lugar)

 `_nombre`: Protegido (convención, uso interno)

 `__nombre`: Privado (name mangling)

 Python no tiene verdadera privacidad, solo convenciones

```
1 class CuentaSegura:
2     def __init__(self, titular, saldo):
3         self.titular = titular           # Público
4         self._password = "1234"         # Protegido
5         self.__saldo = saldo            # Privado
6
7     def consultar_saldo(self, password):
8         if password == self._password:
9             return f"Saldo: ${self.__saldo}"
10        return "Contraseña incorrecta"
11
12    def depositar(self, monto):
13        self.__saldo += monto
14
15 cuenta = CuentaSegura("Ana", 1000)
16 print(cuenta.consultar_saldo("1234"))
17 # print(cuenta.__saldo) # ¡Error! Atributo privado
```


➔ Salida

Saldo: \$1000

! Los atributos privados usan "name mangling": `_NombreClase__atributo`

¿Qué es @property?

Permite crear atributos que se comportan como métodos. Controla el acceso y modificación de atributos.

```
1 class Temperatura:
2     def __init__(self, celsius):
3         self._celsius = celsius
4
5     @property
6     def celsius(self):
7         return self._celsius
8
9     @property
10    def fahrenheit(self):
11        return (self._celsius * 9/5) + 32
12
13    @celsius.setter
14    def celsius(self, valor):
15        if valor < -273.15:
16            raise ValueError("Temperatura bajo cero absoluto")
17        self._celsius = valor
```

```
1 temp = Temperatura(25)
2
3 # Acceder como atributo (no método)
4 print(f"Celsius: {temp.celsius} C ")
5 print(f"Fahrenheit: {temp.fahrenheit} F ")
6
7 # Modificar con validación
8 temp.celsius = 30
9 print(f"Nueva temp: {temp.celsius} C ")
10
11 # temp.celsius = -300 # ¡Error! ValueError
```

➔ Salida

```
Celsius: 25°C
Fahrenheit: 77.0°F
Nueva temp: 30°C
```

¿Qué son?

Métodos con doble guion bajo (`--método--`) que Python llama automáticamente en situaciones específicas.

Métodos comunes:

 `--init--`: Constructor

 `--str--`: Representación legible (para usuarios)

 `--repr--`: Representación técnica (para desarrolladores)

`--eq--`: Igualdad (`==`)

 `--add--`: Suma (`+`)

 `--len--`: Longitud (`len()`)



```
1 class Libro:
2     def __init__(self, titulo, autor, año):
3         self.titulo = titulo
4         self.autor = autor
5         self.año = año
6
7     def __str__(self):
8         # Para usuarios (print)
9         return f'"{self.titulo}" por {self.autor}'
10
11     def __repr__(self):
12         # Para desarrolladores (consola)
13         return f"Libro('{self.titulo}', '{self.autor}', {self.año})"
14
15 libro = Libro("Cien años de soledad", "García Márquez", 1967)
16
17 print(libro)          # Llama __str__
18 print(repr(libro))    # Llama __repr__
```

➔ Salida

```
Cien años de soledad"por García Márquez  
Libro('Cien años de soledad', 'García Márquez', 1967)
```

💡 `__str__` para usuarios, `__repr__` para debug

Más métodos especiales

```
1 class Punto:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __str__(self):
7         return f"({self.x}, {self.y})"
8
9     def __add__(self, otro):
10        return Punto(self.x + otro.x, self.y + otro.y)
11
12    def __eq__(self, otro):
13        return self.x == otro.x and self.y == otro.y
14
15 p1 = Punto(1, 2)
16 p2 = Punto(3, 4)
17 p3 = p1 + p2  # Llama __add__
18
19 print(f"p1 + p2 = {p3}")
20 print(f"p1 == p2: {p1 == p2}")  # Llama __eq__
```

➡ Salida

```
p1 + p2 = (4, 6)
p1 == p2: False
```

i Los métodos especiales hacen que tus objetos se comporten como tipos nativos

¿Qué es herencia?

Mecanismo que permite crear clases nuevas basadas en clases existentes, heredando sus atributos y métodos.

```
1 class Animal:
2     def __init__(self, nombre):
3         self.nombre = nombre
4
5     def hacer_sonido(self):
6         return "Hace un sonido"
7
8     def dormir(self):
9         return f"{self.nombre} está durmiendo"
10
11 # Perro hereda de Animal
12 class Perro(Animal):
13     def hacer_sonido(self):
14         return "¡Guau guau!"
15
16 # Gato hereda de Animal
17 class Gato(Animal):
```

```
1 perro = Perro("Max")
2 gato = Gato("Luna")
3
4 print(perro.hacer_sonido()) # Método sobrescrito
5 print(gato.hacer_sonido()) # Método sobrescrito
6
7 print(perro.dormir())      # Método heredado
8 print(gato.dormir())      # Método heredado
```

➔ Salida

```
¡Guau guau!
¡Miau miau!
Max está durmiendo
Luna está durmiendo
```

super(): Llamar a la clase padre ↑

```
1 class Vehiculo:
2     def __init__(self, marca, modelo):
3         self.marca = marca
4         self.modelo = modelo
5
6     def descripcion(self):
7         return f"{self.marca} {self.modelo}"
8
9 class Coche(Vehiculo):
10     def __init__(self, marca, modelo, puertas):
11         super().__init__(marca, modelo) # Llama al padre
12         self.puertas = puertas
13
14     def descripcion(self):
15         # Reutilizar método del padre y extenderlo
16         return f"{super().descripcion()} - {self.puertas}
17         puertas"
18
19 coche = Coche("Toyota", "Corolla", 4)
20 print(coche.descripcion())
```

➡ Salida

```
Toyota Corolla - 4 puertas
```

💡 `super()` evita repetir código y facilita el mantenimiento

¿Qué es @classmethod?

Método que recibe la clase (cls) como primer argumento, no la instancia. Puede acceder a atributos de clase.

```
1 class Producto:
2     iva = 0.16 # Atributo de clase
3
4     def __init__(self, nombre, precio_base):
5         self.nombre = nombre
6         self.precio_base = precio_base
7
8     @classmethod
9     def cambiar_iva(cls, nuevo_iva):
10         cls.iva = nuevo_iva
11
12     def precio_final(self):
13         return self.precio_base * (1 + Producto.iva)
14
15 # Cambiar IVA para todos los productos
16 Producto.cambiar_iva(0.18)
```

```
1 # Crear productos después del cambio de IVA
2 p1 = Producto("Laptop", 1000)
3 p2 = Producto("Mouse", 20)
4
5 print(f"{p1.nombre}: ${p1.precio_final():.2f}")
6 print(f"{p2.nombre}: ${p2.precio_final():.2f}")
7
8 # Cambiar IVA nuevamente
9 Producto.cambiar_iva(0.20)
10 print(f"\nNuevo precio {p1.nombre}: ${p1.precio_final():.2f}")
```

➔ Salida

Laptop: \$1180.00

Mouse: \$23.60

Nuevo precio Laptop: \$1200.00

@classmethod como constructor alternativo

```
1 class Fecha:
2     def __init__(self, dia, mes, año):
3         self.dia = dia
4         self.mes = mes
5         self.año = año
6
7     @classmethod
8     def desde_string(cls, fecha_str):
9         # Constructor alternativo desde string
10        dia, mes, año = map(int, fecha_str.split('/'))
11        return cls(dia, mes, año)
12
13    def __str__(self):
14        return f"{self.dia:02d}/{self.mes:02d}/{self.año}"
15
16 fecha1 = Fecha(15, 10, 2025)
17 fecha2 = Fecha.desde_string("25/12/2025")
18
19 print(fecha1)
20 print(fecha2)
```

➡ Salida

15/10/2025

25/12/2025

💡 @classmethod es útil para constructores alternativos

¿Qué es @staticmethod?

Método que **no** recibe ni self ni cls. Es una función regular agrupada en la clase por organización.

```
1 class Matematicas:
2     @staticmethod
3     def es_par(numero):
4         return numero % 2 == 0
5
6     @staticmethod
7     def factorial(n):
8         if n <= 1:
9             return 1
10        return n * Matematicas.factorial(n - 1)
11
12    @staticmethod
13    def es_primo(n):
14        if n < 2:
15            return False
16        for i in range(2, int(n ** 0.5) + 1):
17            if n % i == 0:
```

```
1 # No necesitan instancia de la clase
2 print(f"¿4 es par? {Matematicas.es_par(4)}")
3 print(f"¿7 es par? {Matematicas.es_par(7)}")
4
5 print(f"Factorial de 5: {Matematicas.factorial(5)}")
6
7 print(f"¿17 es primo? {Matematicas.es_primo(17)}")
8 print(f"¿18 es primo? {Matematicas.es_primo(18)}")
```

➔ Salida

```
¿4 es par? True
¿7 es par? False
Factorial de 5: 120
¿17 es primo? True
¿18 es primo? False
```

Comparación: Métodos

Tipo	Primer arg	Acceso a	Uso
Instancia	<code>self</code>	Instancia	Operaciones del objeto
Clase	<code>cls</code>	Clase	Config/Constructores
Estático	-	Nada	Utilidades agrupadas

 Usa el tipo de método según lo que necesites acceder

Objetivo

Crear un sistema completo usando POO para gestionar una biblioteca con libros, usuarios y préstamos.

Funcionalidades:

-  Gestión de libros (agregar, buscar, listar)
-  Gestión de usuarios
-  Sistema de préstamos y devoluciones
-  Historial de préstamos
-  Validaciones y restricciones

 Aplicaremos todos los conceptos de POO



```
1 class Libro:
2     total_libros = 0  # Atributo de clase
3
4     def __init__(self, titulo, autor, isbn, copias=1):
5         self.titulo = titulo
6         self.autor = autor
7         self.isbn = isbn
8         self.copias_totales = copias
9         self.copias_disponibles = copias
10        Libro.total_libros += 1
11
12    def __str__(self):
13        return f'"{self.titulo}" por {self.autor}'
14
15    def __repr__(self):
16        return f'Libro(\'{self.titulo}\', \' {self.autor}\', \' {self.isbn}\')'
17
18    def esta_disponible(self):
19        return self.copias_disponibles > 0
```

Proyecto: Clase Usuario

```
1 class Usuario:
2     def __init__(self, nombre, id_usuario):
3         self.nombre = nombre
4         self.id_usuario = id_usuario
5         self._libros_prestados = []    # Protegido
6         self.max_prestamos = 3
7
8     def __str__(self):
9         return f"{self.nombre} (ID: {self.id_usuario})"
10
11     def puede_prestar(self):
12         return len(self._libros_prestados) < self.max_prestamos
13
14     @property
15     def cantidad_prestamos(self):
16         return len(self._libros_prestados)
17
18     def agregar_prestamo(self, libro):
19         self._libros_prestados.append(libro)
20
21     def devolver_libro(self, libro):
22         self._libros_prestados.remove(libro)
```

Proyecto: Clase Prestamo

```
1 from datetime import datetime, timedelta
2
3 class Prestamo:
4     def __init__(self, libro, usuario, dias=14):
5         self.libro = libro
6         self.usuario = usuario
7         self.fecha_prestamo = datetime.now()
8         self.fecha_devolucion = self.fecha_prestamo + timedelta(
9             days=dias)
10         self.devuelto = False
11
12     def __str__(self):
13         estado = "Devuelto" if self.devuelto else "Activo"
14         return f"{self.libro} - {self.usuario} [{estado}]"
15
16     def esta_vencido(self):
17         if self.devuelto:
18             return False
19         return datetime.now() > self.fecha_devolucion
20
21     def marcar_devuelto(self):
22         self.devuelto = True
```

Proyecto: Clase Biblioteca

```
1 class Biblioteca:
2     def __init__(self, nombre):
3         self.nombre = nombre
4         self._catalogo = []           # Libros
5         self._usuarios = []          # Usuarios registrados
6         self._prestamos = []         # Historial de préstamos
7
8     def agregar_libro(self, libro):
9         self._catalogo.append(libro)
10        print(f"    Libro agregado: {libro}")
11
12    def registrar_usuario(self, usuario):
13        self._usuarios.append(usuario)
14        print(f"    Usuario registrado: {usuario}")
15
16    def buscar_libro(self, titulo):
17        for libro in self._catalogo:
18            if titulo.lower() in libro.titulo.lower():
19                return libro
20        return None
```

(Continuación en siguiente slide...)


```
1 def prestar_libro(self, titulo, id_usuario):
2     # Buscar libro
3     libro = self.buscar_libro(titulo)
4     if not libro:
5         return "      Libro no encontrado"
6
7     if not libro.esta_disponible():
8         return f"      No hay copias de {libro} disponibles"
9
10    # Buscar usuario
11    usuario = None
12    for u in self._usuarios:
13        if u.id_usuario == id_usuario:
14            usuario = u
15            break
16
17    if not usuario:
18        return "      Usuario no registrado"
```

(Continuación en siguiente slide...)



```
1         if not usuario.puede_prestar():
2             return f"        {usuario.nombre} alcanzó el límite de
préstamos"
3
4         # Realizar préstamo
5         libro.copias_disponibles -= 1
6         usuario.agregar_prestamo(libro)
7         prestamo = Prestamo(libro, usuario)
8         self._prestamos.append(prestamo)
9
10        dias = (prestamo.fecha_devolucion - prestamo.
fecha_prestamo).days
11        return f"        Préstamo exitoso. Devolver antes de: {
prestamo.fecha_devolucion.strftime('%d/%m/%Y')} ({dias} días
)"
```

Proyecto: Biblioteca - Devolución

```
1  def devolver_libro(self, titulo, id_usuario):
2      # Buscar préstamo activo
3      for prestamo in self._prestamos:
4          if (prestamo.libro.titulo == titulo and
5              prestamo.usuario.id_usuario == id_usuario and
6              not prestamo.devuelto):
7
8              # Procesar devolución
9              prestamo.marcar_devuelto()
10             prestamo.libro.copias_disponibles += 1
11             prestamo.usuario.devolver_libro(prestamo.libro)
12
13             vencido = " (¡VENCIDO!)" if prestamo.
esta_vencido() else ""
14             return f"        Libro devuelto{vencido}"
15
16     return "        No se encontró préstamo activo"
```

```
1  def listar_disponibles(self):
2      print("\n=== LIBROS DISPONIBLES ===")
3      for libro in self._catalogo:
4          if libro.esta_disponible():
5              print(f"        {libro} - {libro.copias_disponibles}
6              copias")
7
8  def prestamos_activos(self):
9      print("\n=== PRÉSTAMOS ACTIVOS ===")
10     for prestamo in self._prestamos:
11         if not prestamo.devuelto:
12             vencido = "        VENCIDO" if prestamo.
13             esta_vencido() else ""
14             print(f"        {prestamo}{vencido}")
15
16 @staticmethod
17 def validar_isbn(isbn):
18     # Validación simple de ISBN-10
19     return len(isbn) == 10 and isbn.isdigit()
```

Proyecto: Uso del sistema >_

```
1 # Crear biblioteca
2 bib = Biblioteca("Biblioteca Central")
3
4 # Agregar libros
5 libro1 = Libro("Cien años de soledad", "García Márquez", "
        1234567890", 2)
6 libro2 = Libro("Don Quijote", "Cervantes", "0987654321", 1)
7
8 bib.agregar_libro(libro1)
9 bib.agregar_libro(libro2)
10
11 # Registrar usuarios
12 user1 = Usuario("Ana García", "U001")
13 user2 = Usuario("Luis Pérez", "U002")
14
15 bib.registrar_usuario(user1)
16 bib.registrar_usuario(user2)
```

(Continuación en siguiente slide...)

```
1 # Realizar préstamos
2 print("\n" + "="*40)
3 print(bib.prestar_libro("Cien años", "U001"))
4 print(bib.prestar_libro("Don Quijote", "U002"))
5 print(bib.prestar_libro("Cien años", "U001"))
6
7 # Listar estado
8 bib.listar_disponibles()
9 bib.prestamos_activos()
10
11 # Devolver libro
12 print("\n" + "="*40)
13 print(bib.devolver_libro("Cien años de soledad", "U001"))
14
15 bib.listar_disponibles()
```

➔ Salida

```
Libro agregado: Cien años de soledad"por García Márquez
Libro agregado: "Don Quijote"por Cervantes
Usuario registrado: Ana García (ID: U001)
Usuario registrado: Luis Pérez (ID: U002)

=====
Préstamo exitoso. Devolver antes de: 04/11/2025 (14 días)
Préstamo exitoso. Devolver antes de: 04/11/2025 (14 días)
No hay copias de Cien años de soledad"por García Márquez disponibles

=== LIBROS DISPONIBLES ===
• Cien años de soledad"por García Márquez - 1 copias

=== PRÉSTAMOS ACTIVOS ===
• Cien años de soledad"por García Márquez - Ana García (ID: U001) [Activo]
• "Don Quijote"por Cervantes - Luis Pérez (ID: U002) [Activo]
```

Alternativa

Sistema para gestionar inventario de productos en una tienda

Clases sugeridas:

-  **Producto:** Base con nombre, precio, stock
-  **ProductoFisico:** Hereda, agrega peso, dimensiones
-  **ProductoDigital:** Hereda, agrega tamaño de archivo
-  **Carrito:** Gestiona lista de compra
-  **Tienda:** Catalogo, ventas, reportes
-  **Cliente:** Datos, historial de compras

Otra alternativa

Sistema para gestionar estudiantes, cursos y calificaciones

Clases sugeridas:



Persona: Clase base con nombre, ID



Estudiante: Hereda, agrega cursos, promedio



Profesor: Hereda, agrega cursos que imparte



Curso: Nombre, código, créditos



Calificación: Estudiante, curso, nota



Universidad: Gestión completa

Principios SOLID

- ✓ **Single Responsibility:** Una clase, una responsabilidad
- ✓ **Open/Closed:** Abierto a extensión, cerrado a modificación
- ✓ **Liskov Substitution:** Subclases sustituibles por su base
- ✓ **Interface Segregation:** Interfaces específicas
- ✓ **Dependency Inversion:** Dependier de abstracciones

Otras prácticas:

- ✓ Nombres descriptivos para clases y métodos
- ✓ Encapsular datos sensibles
- ✓ Preferir composición sobre herencia cuando sea apropiado
- ✓ Documentar clases con docstrings

Documentación con docstrings

```
1 class CuentaBancaria:
2     """
3     Representa una cuenta bancaria básica.
4
5     Attributes:
6         titular (str): Nombre del titular de la cuenta
7         saldo (float): Saldo actual de la cuenta
8
9     Methods:
10         depositar(monto): Deposita dinero en la cuenta
11         retirar(monto): Retira dinero de la cuenta
12     """
13
14     def __init__(self, titular, saldo=0):
15         """Inicializa una cuenta bancaria."""
16         self.titular = titular
17         self.saldo = saldo
```

(Continuación en siguiente slide...)

```
1 def depositar(self, monto):
2     """
3     Deposita dinero en la cuenta.
4
5     Args:
6         monto (float): Cantidad a depositar
7
8     Returns:
9         str: Mensaje de confirmación
10
11     Raises:
12         ValueError: Si el monto es negativo
13     """
14     if monto < 0:
15         raise ValueError("El monto no puede ser negativo")
16     self.saldo += monto
17     return f"Depósito exitoso. Saldo: ${self.saldo}"
```

¿Qué aprendimos hoy?

- ✓ Conceptos de POO: clases y objetos
- ✓ Constructor `__init__` y atributos
- ✓ Métodos de instancia, clase y estáticos
- ✓ Encapsulación y convenciones de privacidad
- ✓ Propiedades con `@property`
- ✓ Métodos especiales: `__str__`, `__repr__`
- ✓ Herencia y `super()`
- ✓ Decoradores: `@classmethod`, `@staticmethod`
- ✓ Proyecto: Sistema de gestión de biblioteca

 ¡POO organiza y potencia tu código!



¡Preguntas!

Gracias por su atención